

GUIA DE C

Da primeira declaracao ao codigo portavel

C17 com notas de C11/C23

Sintaxe + exemplos + biblioteca padrao + boas praticas

Edicao de estudo | Setembro de 2026

Como usar este livro

Este livro-resumo foi feito para estudo e consulta. Leia em ordem na primeira passagem e depois use cada capítulo como referencia. Ele cobre o núcleo, as famílias de funções e os componentes padrão; APIs de sistema e bibliotecas de terceiros ficam fora do escopo.

Legenda mental

- Sintaxe e a forma válida de escrever.
- Semântica e o significado e o momento de execução.
- Biblioteca padrão adiciona tipos e funções portáveis.
- Comportamento indefinido significa que o padrão não promete resultado.

Escopo: abrangente como livro-resumo, mas não substitui a especificação normativa nem a documentação exata do compilador.

Mapa do livro

- 1. Como C funciona
- 2. Tokens, tipos e declarações
- 3. Expressões e operadores
- 4. Controle de fluxo
- 5. Funções
- 6. Arrays, strings e ponteiros
- 7. Tipos compostos
- 8. Memória e duração
- 9. Pré-processador e módulos por arquivo
- 10. Entrada, saída e arquivos
- 11. Erros e diagnóstico
- 12. Biblioteca padrão: cabeçalhos
- 13. Biblioteca padrão: funções
- 14. Concorrência e portabilidade
- 15. Checklist final

1. Como C funciona

C é compilada, procedural e próxima do hardware. O fonte passa por pré-processamento, compilação, montagem e ligação.

Primeiro programa

```
#include <stdio.h>

int main(void) {
    puts("Ola, C!");
    return 0;
}
```

- Diretivas # são processadas antes da compilação.
- Declarações apresentam nomes e tipos; instruções executam ações.
- Blocos usam chaves; instruções normalmente terminam em ponto e vírgula.
- main retorna int; zero sinaliza sucesso.

Build recomendado

```
cc -std=c17 -Wall -Wextra -Wpedantic programa.c -o programa
```

2. Tokens, tipos e declaracoes

O tipo define representacao, operacoes e conversoes. Tokens incluem palavras-chave, identificadores, literais, operadores e pontuacao.

Familia	Tipos
Inteiros	char, short, int, long, long long; signed/unsigned
Reais	float, double, long double
Logico	_Bool; bool via <stdbool.h> em C17
Sem valor	void; retorno vazio ou ponteiro generico
Derivados	ponteiro, array, funcao, struct, union, enum

```
const unsigned long limite = 1000UL;
double taxa = 2.5e-3;
char letra = 'A';
size_t bytes = sizeof taxa;
typedef unsigned long Id;
```

- const restringe modificacao; volatile indica mudanca externa.
- auto e armazenamento automatico, nao deducao de tipo.
- stdint.h oferece int32_t, uint64_t e familias de largura conhecida.
- Uma declaracao pode conter especificadores de armazenamento: extern, static, _Thread_local.

3. Expressoes e operadores

Expressoes calculam valores e podem causar efeitos colaterais. Precedencia agrupa; ordem de avaliacao e uma regra separada.

Grupo	Operadores e regra
Aritmetica	+ - * / %; divisao inteira descarta fracao
Comparacao	== != < <= > >=
Logica	&& !; curto-circuito em && e
Bits	& ^ ~ << >> em inteiros
Atribuicao	= += -= *= /= %= e formas de bits
Acesso	[] () . ->
Outros	++ -- ?: , sizeof; use parenteses quando houver duvida

```
double media = (double)soma / quantidade;
unsigned mascara = 1u &lt;&lt; 5;
int maior = a &gt; b ? a : b;
if (p != NULL &amp;&amp; p-&gt;ativo) usar(p);
```

Em resumo: Nao combine efeitos colaterais ambiguos. Separe passos e torne conversoes com perda explicitas.

4. Controle de fluxo

Selecao escolhe caminhos; repeticao executa blocos enquanto uma condicao permitir.

```
if (nota >= 7) puts("aprovado");  
else if (nota >= 5) puts("recuperacao");  
else puts("reprovado");
```

```
switch (opcao) {  
case 1: iniciar(); break;  
case 2: pausar(); break;  
default: puts("invalida");  
}
```

```
for (size_t i=0; i<n; ++i) processar(i);  
while (condicao) atualizar();  
do { ler(); } while (repetir);
```

- break sai do loop ou switch interno.
- continue avanca a iteracao.
- return encerra a funcao.
- goto salta para rotulo na mesma funcao; pode centralizar limpeza.

5. Funcoes

Funcoes possuem retorno, nome, parametros e corpo. Prototipos permitem verificar chamadas antes da definicao.

```
double media(const int v[], size_t n);

double media(const int v[], size_t n) {
    long soma = 0;
    for (size_t i=0; i<n; ++i) soma += v[i];
    return n ? (double)soma/n : 0.0;
}

typedef int (*Comparador)(const void*, const void*);
```

- Argumentos sao passados por valor; use ponteiro para alterar o objeto do chamador.
- Parametro de array ajusta para ponteiro; passe tambem o tamanho.
- static limita uma funcao ao arquivo.
- Funcoes variadicas usam ... e stdarg.h.
- inline tem regras de ligacao; use com cuidado em cabecalhos.

6. Arrays, strings e ponteiros

Arrays sao contiguos. Ponteiros guardam enderecos. Strings C sao char terminados pelo byte zero.

```
int v[4] = {10,20,30,40};
int *p = v;
size_t n = sizeof v / sizeof v[0];
char nome[32] = "Ada";
size_t len = strlen(nome);
int m[2][3] = {{1,2,3},{4,5,6}};
```

- Aritmetica de ponteiro so vale no mesmo array ou um apos o fim.
- sizeof(array) so da o total enquanto o valor ainda e array.
- Literais de string nao devem ser modificados.
- Acesso fora dos limites e comportamento indefinido.
- restrict promete ausencia relevante de sobreposicao.

7. Tipos compostos

struct modela registros, union compartilha armazenamento, enum nomeia constantes e bit-fields compactam campos com portabilidade limitada.

```
typedef struct { char nome[40]; int idade; } Pessoa;  
enum Estado { PARADO, RODANDO, ERRO };  
union Valor { long inteiro; double real; };  
Pessoa p = {.nome="Ada", .idade=36};
```

- struct pode conter padding; memoria bruta nao e formato portavel.
- Acompanhe qual membro de union esta ativo.
- enum gera constantes inteiras; representacao pode variar.
- Tipos incompletos permitem APIs opacas e listas autorreferentes.

8. Memoria e duracao

Objetos podem ter duracao automatica, estatica, de thread ou dinamica. Propriedade deve ser explicita.

```
int *criar(size_t n) {
    if (n > SIZE_MAX / sizeof(int)) return NULL;
    return calloc(n, sizeof(int));
}
int *v = criar(100);
if (!v) { /* falha */ }
/* usar */
free(v);
v = NULL;
```

Funcao	Contrato resumido
malloc(n)	n bytes nao inicializados
calloc(q,t)	q objetos; bytes zerados
realloc(p,n)	pode mover; preserve p ate confirmar
free(p)	libera; aceita NULL

Em resumo: Evite vazamento, double free, use-after-free e ponteiro pendente. Defina quem aloca, possui e libera.

9. Pre-processor e modulos por arquivo

O pre-processor inclui texto e expande macros. A interface fica em .h; a implementacao em .c.

```
#ifndef MINHA_BIB_H
#define MINHA_BIB_H
#include <stddef.h>
#define ARRAY_LEN(a) (sizeof(a)/sizeof((a)[0]))
int somar(const int *v, size_t n);
#endif
```

- Use include guards; #pragma once e comum, mas nao universal.
- Parentetize macros e nao avalie argumento duas vezes.
- #if/#ifdef controlam compilacao condicional.
- # e ## geram string e concatenam tokens.
- Prefira funcao inline a macro de calculo.

10. Entrada, saida e arquivos

stdio.h oferece fluxos, formatacao e arquivos. Verifique cada retorno.

```
FILE *f = fopen("dados.txt","r");
if (!f) { perror("fopen"); return 1; }
char linha[256];
while (fgets(linha,sizeof linha,f)) fputs(linha,stdout);
if (ferror(f)) perror("leitura");
if (fclose(f)!=EOF) perror("fclose");
```

Familia	Funcoes
Fluxos	stdin, stdout, stderr, fopen, freopen, fclose, fflush
Formatada	printf, fprintf, snprintf, scanf, fscanf, sscanf
Texto	fgetc, fgets, fputc, fputs, ungetc
Binaria	fread, fwrite
Posicao	fseek, ftell, rewind, fgetpos, fsetpos
Estado	feof, ferror, clearerr, perror, remove, rename, tmpfile

Em resumo: gets foi removida. Limite %s em scanf. Para entrada robusta, use fgets e depois converta.

11. Erros e diagnostico

C nao tem excecoes. APIs usam retornos, errno, parametros de saida ou estados da aplicacao.

```
errno = 0;
char *fim;
long x = strtol(texto,&fim,10);
if (fim==texto || *fim!='\0' || errno==ERANGE) {
    /* invalido */
}
assert(indice < tamanho);
```

- errno so vale quando a funcao documenta e sinaliza falha.
- assert verifica invariantes internas, nao entrada hostil.
- Ative avisos altos, sanitizers, testes e analise estatica.
- Documente contratos, intervalos e efeitos colaterais.

12. Biblioteca padrao: cabecalhos

Este mapa cobre os cabecalhos de C moderno. Alguns dependem da versao ou implementacao.

Cabecalho	Conteudo
assert.h	assert
complex.h / tgmth.h	complexos e matematica generica
ctype.h / wctype.h	classificacao de caracteres
errno.h	errno e codigos
fcntl.h / float.h	ambiente e limites reais
inttypes.h / stdint.h	inteiros exatos, formatos e conversoes
limits.h	limites inteiros
locale.h	localeconv, setlocale
math.h	matematica real
setjmp.h / signal.h	retorno nao local e sinais
stdalign.h / stddef.h	alinhamento, size_t, ptrdiff_t, offsetof
stdarg.h	variadic
stdatomic.h	atomicos
stdbool.h	bool em C99-C17
stdio.h	entrada e saida
stdlib.h	memoria, conversao, busca, processo
stdnoreturn.h	_Noreturn em C11/C17
string.h	bytes e strings
threads.h	threads e sincronizacao
time.h	tempo
uchar.h / wchar.h	Unicode de largura fixa e caracteres largos
iso646.h	grafias alternativas

13. Biblioteca padrao: funcoes

Aprenda as familias e consulte o contrato completo quando dominios, complexidade ou portabilidade importarem.

Area	Nomes principais
Strings	strlen, strcpy, strncpy, strcat, strcmp, strchr, strstr, strtok, strspn
Memoria	memcpy, memmove, memset, memcmp, memchr
Conversao	strtol, strtoul, strtod, strtod; atoi e menos robusta
Algoritmos	qsort, bsearch
Processo	exit, _Exit, abort, atexit, system, getenv
Aleatorio	rand, srand; nao criptografico
Matematica	sqrt, pow, exp, log, sin, cos, atan2, ceil, floor, round, fabs, fmod, isfinite, isnan
Caracteres	isalnum, isalpha, isdigit, isspace, toupper, tolower
Tempo	time, difftime, clock, mktime, localtime, gmtime, strftime, timespec_get
Atomicos	atomic_load/store/exchange, compare_exchange, fetch_*
Threads	thrd_create/join, mtx_*, cnd_*, call_once, tss_*
Variadicas	va_start, va_arg, va_copy, va_end

Em resumo: strcpy/strcat nao recebem capacidade; strncpy tem armadilhas. memcpy exige nao sobreposicao; use memmove quando sobrepo.

14. Concorrencia e portabilidade

Data race em objeto nao atomico e comportamento indefinido. Portabilidade exige evitar suposicoes sobre maquina e compilador.

```
#include <threads.h>
#include <stdatomic.h>
atomic_int total = 0;
int tarefa(void *arg) {
    atomic_fetch_add(&total, *(int*)arg);
    return 0;
}
```

- Mutex protege invariantes; condicao deve ser testada em loop.
- Atomicos incluem uma ordem de memoria no contrato.
- Manipulador de sinal so pode executar operacoes permitidas.
- Nao estoure signed, use variavel indeterminada ou shift invalido.
- Considere endian, largura, alinhamento, signedness de char e ponto flutuante.
- Use limits.h, stdint.h e float.h em vez de suposicoes.

15. Checklist final

C sustentavel limita responsabilidades, valida limites e torna propriedade visivel.

Situacao	Padrao
Buffer	ponteiro + capacidade/comprimento em size_t
Recurso	um proprietario e caminho de limpeza
API opaca	struct incompleta no .h; definicao no .c
Erro	retorno documentado e diagnostico
Estado global	evitar; encapsular e sincronizar
Build	padrao fixo, avisos, testes, sanitizers

Em resumo: Cada acesso deve apontar para objeto vivo, dentro dos limites, com tipo e alinhamento validos; cada recurso adquirido precisa de liberacao.

Proximos passos

Reescreva os exemplos, preveja resultados antes de executar e construa programas pequenos completos. Quando um contrato exato importar, consulte uma referencia atualizada da versao escolhida.

- Revisar tipos e expressoes.
- Praticar controle e funcoes.
- Dominar memoria e tempo de vida.
- Aplicar biblioteca padrao.
- Separar um projeto em arquivos.
- Adicionar testes, avisos e analisadores.